外送平台系統設計

你現在被任命為知名外送平台（如 foodpanda）的軟體架構師。公司的核心業務包含顧客、外送員與合作餐廳，且未來的發展變化極快。為了確保系統長遠具備擴充性、高內聚與低耦合，主管要求你依據以下最新的業務需求，進行基礎的系統建模與核心段落實作：

1. 平台使用者 (Users)：
    - 系統中有「顧客 (Customer)」、「外送員 (Rider)」與「餐廳 (Restaurant)」三種主要角色。
    - 所有角色皆具備共用的基本資訊（ID、名稱、聯絡電話），並且都必須實作一個名為 `displayProfile()` 的方法來向系統回報當前角色狀態。
2. 菜單與餐點 (Menu & Items)：
    - 每家「餐廳」擁有「專屬菜單 (Menu)」，菜單內包含多樣「餐點品項 (MenuItem)」（如：大麥克、冰炫風等）。
    - 如果一家餐廳退出平台，其專屬菜單將一併被系統註銷。但存在菜單中的具體「餐點品項」，因爲歷史訂單也可能會參照，所以不一定會跟著消失。
3. 訂單與結帳 (Orders & Tracks)：
    - 顧客可以開立一張「訂單 (Order)」，每張訂單中包含多個「訂單明細 (OrderItem)」。每個明細會註明一份具體的「餐點品項」。
    - 如果顧客中途取消並刪除該筆訂單，其附屬的訂單明細也應隨之銷毀。
    - 訂單的核心狀態（如總金額、運送狀態）不能被外界直接竄改，必須透過專門的方法才能變更，以確保商業邏輯安全。
    - 為了整合第三方地圖與金流 API，系統規定：外送員與訂單都必須具備「被追蹤位置（`Trackable`）」的特性；而訂單本身還必須具備「可被支付結帳（`Payable`）」的特性。

請依據上述情境，完成以下兩項開發任務：

(1) 繪製 UML 類別圖 (50%) 請利用你熟悉的繪圖工具或手繪，畫出符合上述業務情境的 UML 類別圖。評分標準將著重於：

1. 結構設計：是否能正確判別出介面 (Interface)、抽象類別 (Abstract Class) 與具體的實體類別。
2. 封裝原則：必須在你的屬性與方法前加上正確的可視性符號（如 `+` 代表公開, `–` 代表私有）。
3. 物件關係：請精準區分並畫出對應的線條關係。例如繼承/實作、組合 (Composition)等。

(2) 情境程式碼實作 (50%) 請使用 Java，搭配你在任務一中設計的類別架構，完成以下「結帳與印出明細」情境的程式碼實作：

實作要求：

1. 建立實例：撰寫主程式（`main` 方法)，並在其中建立一名顧客實例，以及一張屬於該顧客的空白訂單（`Order`）。
2. 購買商品：建立至少兩項不同的餐點品項（`MenuItem`），並指定數量將它們封裝成訂單明細（`OrderItem`），最後加入到上述的訂單中。
3. 列出結帳清單與總和：
    - 請在 `Order` 類別中設計並實作一個能印出結帳清單的方法（例如 `printReceipt()`）。
    - 該方法必須能夠透過迴圈，依序印出該訂單內所有的商品名稱、單價與各自購買數量。

- 迴圈結束後，結算並印出這筆訂單的總金額 (Total Amount)。

Hint

```
public class FoodpandaCheckoutApp {
    public static void main(String[] args) {
        Customer customer = new Customer("王大明");
        Order order = new Order("ORD-1001A");

        MenuItem item1 = new MenuItem("大麥克套餐", 150.0);
        MenuItem item2 = new MenuItem("麥克雞塊 (10塊)", 105.0);

        order.addItem(new OrderItem(item1, 2));
        order.addItem(new OrderItem(item2, 1));

        System.out.println("顧客：" + customer.getName() + " 準備結帳...");
        order.printReceipt();
    }
}
```

👩‍🏫 教師參考解答 (Reference Answer)

參考任務一：UML 設計圖

```
classDiagram
    direction TB

    class Trackable {
        <<interface>>
        +getLocation() String
    }

    class Payable {
        <<interface>>
        +processPayment(amount) boolean
    }

    class PlatformUser {
        <<abstract>>
        -String userId
        -String name
        -String phone
        +displayProfile()* void
        +getPhoneNumber() String
    }
```

```
class Customer {
    -String address
    +displayProfile() void
}

class Rider {
    -String vehicleType
    -String currentLocation
    +displayProfile() void
    +getLocation() String
}

class Restaurant {
    -double rating
    +displayProfile() void
}

class Order {
    -String orderId
    -double totalAmount
    -String status
    +processPayment(amount) boolean
    +getLocation() String
    +updateStatus(newStatus) void
}

class OrderItem {
    -int quantity
}

class Menu {
    -String category
}

class MenuItem {
    -String itemName
    -double price
}

Trackable <|.. Rider
Trackable <|.. Order
Payable <|.. Order

PlatformUser <|-- Customer
PlatformUser <|-- Rider
PlatformUser <|-- Restaurant
```

```
    Restaurant "1" *-- "1" Menu : 組合(共存亡)
    Menu "1" o-- "*" MenuItem : 聚合(菜單註銷,菜色依然可存)

    Order "1" *-- "*" OrderItem : 組合(訂單取消,明細作廢)
    OrderItem "*" o-- "1" MenuItem : 聚合
```

```java
import java.util.ArrayList;
import java.util.List;

// --- 基礎模型宣告 ---

class MenuItem {
    private String itemName;
    private double price;

    public MenuItem(String name, double price) {
        this.itemName = name;
        this.price = price;
    }
    public String getItemName() { return itemName; }
    public double getPrice() { return price; }
}

class OrderItem {
    private MenuItem item;
    private int quantity;

    public OrderItem(MenuItem item, int quantity) {
        this.item = item;
        this.quantity = quantity;
    }
    public MenuItem getItem() { return item; }
    public int getQuantity() { return quantity; }
    // 計算此明細的小計
    public double getSubtotal() { return item.getPrice() * quantity; }
}

class Order {
    private String orderId;
    private List<OrderItem> items;
    private double totalAmount; // 總金額封裝在內部計算

    public Order(String id) {
        this.orderId = id;
        this.items = new ArrayList<>();
```

```java
        this.totalAmount = 0.0;
    }

    public void addItem(OrderItem orderItem) {
        items.add(orderItem);
    }

    // 負責印出收據與結算總額的方法
    public void printReceipt() {
        System.out.println("=== 結帳清單 [" + orderId + "] ===");
        totalAmount = 0.0; // 初始化總金額

        for (OrderItem orderItem : items) {
            MenuItem item = orderItem.getItem();
            double subtotal = orderItem.getSubtotal();
            System.out.printf("- %s (單價: $%.0f) x %d = $%.0f\n",
                item.getItemName(), item.getPrice(),
orderItem.getQuantity(), subtotal);

            // 累加結算
            totalAmount += subtotal;
        }
        System.out.println("-------------------------");
        System.out.printf("總和 (Total): $%.0f\n", totalAmount);
    }
}

class Customer {
    private String name;
    public Customer(String name) { this.name = name; }
    public String getName() { return name; }
}

// --- 主程式情境實作 ---

public class FoodpandaCheckoutApp {
    public static void main(String[] args) {

        // 1. 建立實例
        Customer customer = new Customer("王大明");
        Order order = new Order("ORD-1001A");

        // 2. 購買商品 (建立餐點與明細並加入訂單)
        MenuItem item1 = new MenuItem("大麥克套餐", 150.0);
        MenuItem item2 = new MenuItem("麥克雞塊 (10塊)", 105.0);
```

```java
        order.addItem(new OrderItem(item1, 2)); // 買兩份大麥克套餐
        order.addItem(new OrderItem(item2, 1)); // 買一份麥克雞塊

        // 3. 列出結帳清單與總和
        System.out.println("顧客：" + customer.getName() + " 準備結帳...");
        order.printReceipt();
    }
}
```